UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA

Facultad de Ingeniería de Producción y Servicios



Curso: Sistemas Operativos

INFORME TÉCNICO

TicketStorm: simulación concurrente de venta de entradas

con el modelo productor-consumidor en C++17

Docente: Ing. José Rafael Delgado Bastidas

Integrantes:

Mejía Rondan, Giovanni Patrick

Guillen Valcarcel, Daniel Josue

Cari Quispe, Juan Manuel

Subia Huaicane, Edson Fabricio

Mengoa Valeriano, Brigitte Noelia

Arequipa, noviembre de 2026

Resumen

Este informe presenta TicketStorm, una simulación en C++17 de un sistema de venta de entradas pensada para estudiar la sincronización de procesos. El programa modela el problema productor-consumidor con un hilo Dispatcher, varios Workers y una cola acotada con mutex y variables de condición.

Además del diseño y la implementación, se documentan el plan de pruebas, los resultados y los errores encontrados en la revisión. Tras corregir un fallo de portabilidad en Windows que impedía usar `--seed`, pasaron las 10 pruebas de integración y las 10 pruebas unitarias.

También se mejoraron los módulos de inventario y logger con un modo atómico, marcas de tiempo y trazas de eventos. Con esas trazas se construyó un visualizador animado que muestra, evento por evento, cómo el sistema operativo coordina los hilos.

Los experimentos confirman que, sin el mutex del contador, el sistema llega a vender más entradas de las disponibles (5 de 20 ejecuciones), mientras que con sincronización la verificación fue correcta en el 100 % de los casos.

Palabras clave: *concurrencia, productor-consumidor, mutex, variables de condición, condición de carrera, C++17.*

Índice general

1. Introducción 1
2. Objetivos 1
2.1. Objetivo general 1
2.2. Objetivos específicos 1
3. Marco teórico 2
3.1. Procesos, hilos y concurrencia 2
3.2. Sección crítica y condición de carrera 2
3.3. Mutex y variables de condición 3
3.4. Productor-consumidor con buffer acotado 3
3.5. Interbloqueo 3
3.6. Verificación de programas concurrentes 4
4. Organización del equipo 4
5. Diseño del sistema 5
5.1. Arquitectura general 5
5.2. Recursos compartidos 5
5.3. Flujo de ejecución 6
5.4. Protocolo de terminación 6
6. Implementación 6
6.1. Cola acotada (RequestQueue) 6
6.2. Dispatcher 7
6.3. Worker 7
6.4. Inventario de entradas 7
6.5. Logger 8
6.6. Configuración y orquestación 8
6.7. Registro de eventos de sincronización 8
6.8. Visualizador animado para la exposición 9
7. Compilación y ejecución 10
7.1. Requisitos y entorno de prueba 10
7.2. Comandos principales 10
7.3. Parámetros y códigos de salida 11
8. Plan de pruebas y resultados 11
8.1. Metodología 11
8.2. Resultados 12
9. Errores detectados, correcciones y mejoras 13
9.1. Truncamiento de long en Windows 13
9.2. Portabilidad del build y de las pruebas 14

9.3. Limitación observada 14
9.4. Control de versiones 14
9.5. Mejoras a los módulos de inventario y logger 16
10. Análisis de resultados 16
10.1. Efecto del mutex en la consistencia 16
10.2. Escalabilidad con el número de Workers 17
10.3. Mutex frente a operaciones atómicas 18
10.4. Resolución del temporizador del SO 19
11. Conclusiones 19
12. Recomendaciones y trabajo futuro 20
Referencias 21
Anexo A. Salida de ejemplo 22
Anexo B. Salida del plan de pruebas 22
Anexo C. Historial de commits 22
Anexo D. Guía rápida para la exposición 23

Índice de figuras

Figura 1. Arquitectura de TicketStorm y relación entre sus componentes 5
Figura 2. Visualizador: W1 tiene el mutex del inventario y W2 y W3 esperan bloqueados 9
Figura 3. Visualizador en el instante de la condición de carrera (modo sin sincronización).. 10
Figura 4. Historial de ramas del repositorio 15
Figura 5. Entradas vendidas en 20 ejecuciones con y sin mutex (stock de 100) 17
Figura 6. Tiempo total de la simulación según el número de Workers 18
Figura 7. Costo por compra según el número de hilos (mediana de 5 repeticiones) 19

Índice de tablas

Tabla 1. División del trabajo por integrante 4
Tabla 2. Recursos compartidos y mecanismo de protección 5
Tabla 3. Opciones de línea de comandos 11
Tabla 4. Códigos de salida del programa 11
Tabla 5. Plan de pruebas y resultados obtenidos 12
Tabla 6. Errores detectados y corrección aplicada 13
Tabla 7. Ramas creadas y propósito de cada una 15
Tabla 8. Mejoras introducidas en los módulos 16
Tabla 9. Tiempo total según el número de Workers (60 peticiones, 50–150 ms) 17
Tabla 10. Costo por compra en modo mutex y atómico (ns por operación) 18

1. Introducción

Hoy casi cualquier sistema que atiende usuarios, desde una web de entradas hasta un banco, recibe muchas solicitudes al mismo tiempo. Si esas solicitudes tocan los mismos datos sin coordinación, aparecen errores difíciles de reproducir, como vender dos veces el mismo asiento.

Según Silberschatz, Galvin y Gagne, en su libro *Operating System Concepts*, publicado por Wiley en el año 2018, la sincronización de procesos existe justamente para mantener la consistencia de los datos compartidos cuando varios hilos se ejecutan de forma concurrente [1].

TicketStorm nace como un proyecto académico para llevar esa teoría a código. El programa simula la venta de un stock limitado de entradas: un hilo productor genera peticiones de compra y varios hilos consumidores las atienden, descontando entradas de un contador compartido.

El informe se organiza así: la sección 2 plantea los objetivos y la sección 3 resume el marco teórico. Las secciones 4 a 7 describen la organización del equipo, el diseño, la implementación y la forma de compilar y ejecutar el sistema.

Luego, la sección 8 detalla el plan de pruebas y sus resultados, la sección 9 explica los errores detectados y cómo se corrigieron, y la sección 10 analiza los datos obtenidos. El documento cierra con conclusiones, recomendaciones, referencias y anexos.

2. Objetivos

2.1. Objetivo general

Diseñar, implementar y validar en C++17 una simulación concurrente de venta de entradas que aplique el modelo productor-consumidor y demuestre, de forma experimental, la necesidad de la exclusión mutua sobre los recursos compartidos.

2.2. Objetivos específicos

- Implementar una cola acotada segura para hilos con `std::mutex` y dos variables de condición.
- Desarrollar un hilo productor (Dispatcher) y N hilos consumidores (Workers) con terminación limpia.
- Proteger el contador de entradas y la consola, y contrastar con una versión sin mutex.
- Diseñar un plan de pruebas automatizado que verifique la consistencia del stock al final.

- Gestionar el código con Git usando ramas temáticas y commits descriptivos.

3. Marco teórico

3.1. Procesos, hilos y concurrencia

Según Tanenbaum y Bos [2], en su libro *Modern Operating Systems* (2015), un hilo es una unidad de ejecución que comparte el espacio de direcciones de su proceso con otros hilos. Esa compartición es lo que los hace ligeros, pero también lo que obliga a coordinarlos.

En TicketStorm todos los hilos viven dentro del mismo proceso y acceden a los mismos objetos: la cola de peticiones, el inventario de entradas y la consola. Por eso cada uno de esos objetos se trata como un recurso compartido que necesita protección.

Como señala Stallings en la novena edición de *Operating Systems: Internals and Design Principles*, publicada en 2018, la concurrencia trae consigo problemas de exclusión mutua, interbloqueo e inanición que el diseñador debe resolver de forma explícita [3].

3.2. Sección crítica y condición de carrera

Una sección crítica es el fragmento de código en el que un hilo accede a datos compartidos. Si dos hilos ejecutan su sección crítica a la vez y el resultado depende del orden en que se intercalan, se produce una condición de carrera.

Según Edsger W. Dijkstra, en su reporte técnico "Cooperating sequential processes", publicado por la Universidad Tecnológica de Eindhoven en 1965, el problema de la exclusión mutua puede resolverse con primitivas de sincronización como los semáforos [4].

El caso típico en TicketStorm es el patrón verificar y luego actuar (*check-then-act*): un Worker comprueba que hay stock suficiente y después lo descuenta. Si otro hilo verifica entre ambos pasos, los dos pueden vender las mismas entradas.

Boehm y Adve [5], en su artículo "Foundations of the C++ concurrency memory model" (2008), explican que en C++ una carrera de datos no solo da resultados incorrectos, sino que provoca comportamiento indefinido. Por eso la versión sin mutex del proyecto se usa solo con fines didácticos.

3.3. Mutex y variables de condición

Según Anthony Williams, en su libro *C++ Concurrency in Action*, publicado por Manning en 2019, `std::mutex` junto con `std::lock_guard` y `std::unique_lock` ofrecen exclusión mutua con liberación automática del cerrojo gracias al patrón RAII [6].

Estas clases forman parte de la biblioteca estándar desde C++11 y están definidas en la norma ISO/IEC 14882:2017, que es la versión del lenguaje que usa el proyecto [7]. Así se puede escribir código concurrente portable sin depender de la API de cada sistema operativo.

Las variables de condición provienen del concepto de monitor. Hoare [8], en su artículo "Monitors: An operating system structuring concept" de 1974, propuso que un hilo pueda esperar dentro del monitor hasta que otro le avise que cierta condición ya se cumple.

Lampson y Redell [9], en su artículo sobre procesos y monitores en Mesa (1980), introdujeron la semántica en la que el hilo despertado debe volver a comprobar la condición. Es la que usa C++, y por eso las esperas del proyecto llevan un predicado contra los despertares espurios.

En la misma línea, Butenhof [10], en *Programming with POSIX Threads* (1997), recomienda que toda espera sobre una variable de condición se haga dentro de un bucle que evalúe el predicado asociado, y nunca con una espera simple.

3.4. Productor-consumidor con buffer acotado

Según Remzi y Andrea Arpaci-Dusseau, en su libro *Operating Systems: Three Easy Pieces*, publicado en 2018, el problema del buffer acotado se resuelve con un mutex y dos condiciones: una que avisa que hay espacio y otra que avisa que hay datos [11].

El productor se bloquea cuando el buffer está lleno y el consumidor cuando está vacío. Ninguno consume CPU mientras espera, a diferencia de una espera activa (*busy waiting*), que revisa la condición una y otra vez dentro de un bucle.

En la obra ya citada [1] también se resalta que el buffer acotado es la base de muchos sistemas reales, como las colas de impresión o los servidores web con un número fijo de hilos trabajadores. TicketStorm reproduce exactamente ese esquema.

3.5. Interbloqueo

Coffman, Elphick y Shoshani [12], en su artículo "System deadlocks" (1971), identificaron cuatro condiciones necesarias para que ocurra un interbloqueo, entre ellas la espera circular. Romper cualquiera de ellas basta para evitarlo.

El proyecto evita la espera circular imponiendo un orden fijo al adquirir cerrojos: primero el de la cola y después el de la consola, nunca al revés. El inventario y la consola nunca se bloquean juntos, así que no se forma ningún ciclo.

3.6. Verificación de programas concurrentes

Según Serebryany e Iskhodzhanov [13], en su artículo "ThreadSanitizer: Data race detection in practice" (2009), la instrumentación dinámica permite detectar carreras de datos reales durante la ejecución. El Makefile del proyecto incluye un objetivo `tsan` con esta herramienta.

Por otro lado, Herlihy y Shavit [14], en *The Art of Multiprocessor Programming* (2008), recuerdan que probar programas concurrentes es difícil porque un error puede aparecer solo en algunas intercalaciones. Por eso las pruebas de TicketStorm repiten ejecuciones.

4. Organización del equipo

El trabajo se repartió por módulos, de forma que cada integrante fuera responsable de un componente con una interfaz clara. La Tabla 1 resume la división acordada.

Tabla 1. División del trabajo por integrante

N.º	Integrante	Módulos	Responsabilidades
1	Mejía Rondan, Giovanni Patrick	`request_queue.hpp/.cpp`, `request.hpp`	Cola acotada con `std::mutex`, condiciones «hay espacio» y «hay trabajo», bandera `closed_` y `notify_all`. Núcleo del productor-consumidor.
2	Guillen Valcarcel, Daniel Josue	`dispatcher.hpp/.cpp`	Hilo productor: genera peticiones aleatorias (usuario y 1 a 4 entradas), maneja el bloqueo con cola llena y cierra la cola al terminar.
3	Cari Quispe, Juan Manuel	`worker.hpp/.cpp`	Hilo consumidor: toma de la cola, simula el procesamiento, confirma o rechaza la compra y lleva estadísticas por Worker.
4	Subia Huaicane, Edson Fabricio	`ticket_inventory.hpp/.cpp`, `logger.hpp/.cpp`	Contador de entradas con mutex propio, versión sin mutex para el contraste y consola protegida con mutex.
5	Mengoa Valeriano, Brigitte Noelia	`config.hpp/.cpp`, `simulation.hpp/.cpp`, `main.cpp`, `Makefile`, `run_tests.sh`	Parámetros por línea de comandos, orquestación de hilos, `join`, resumen final y verificación, compilación y plan de pruebas.

Fuente: elaboración propia.

Con esta división los módulos se integran a través de pocas dependencias. Por ejemplo, el Dispatcher y los Workers solo conocen la interfaz pública de `RequestQueue` y no necesitan saber cómo está implementada la cola por dentro.

5. Diseño del sistema

5.1. Arquitectura general

TicketStorm sigue una arquitectura productor-consumidor clásica. La Figura 1 muestra los componentes principales y cómo se comunican: el Dispatcher inserta peticiones en la cola y los Workers las extraen para intentar la compra en el inventario.

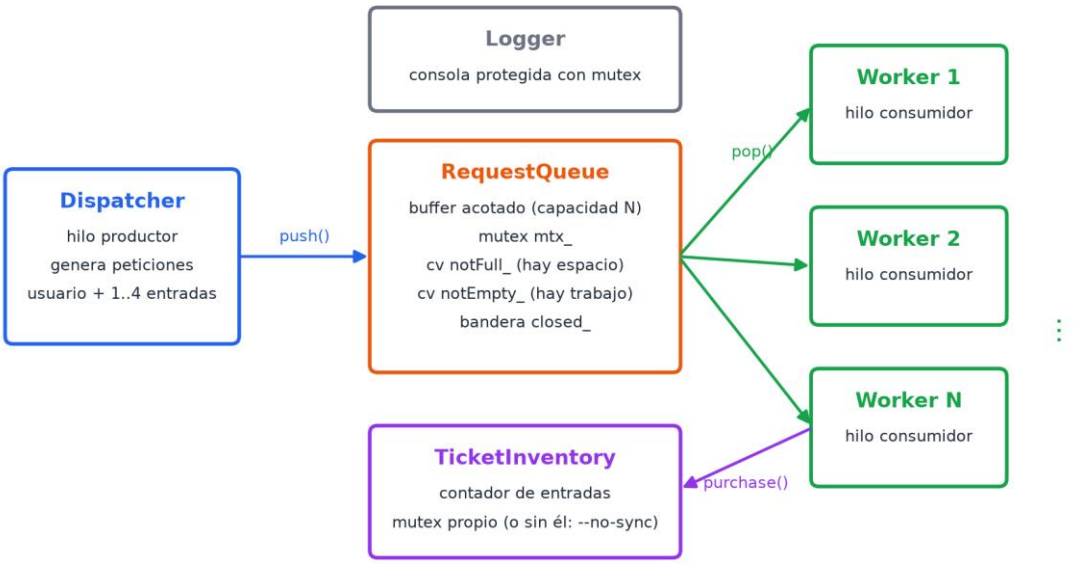


Figura 1. Arquitectura de TicketStorm y relación entre sus componentes

Fuente: elaboración propia.

Cada flecha de la figura corresponde a una llamada que adquiere el cerrojo del recurso de destino. El Logger lo usan todos los hilos, por lo que se dibuja aparte para no recargar el diagrama.

5.2. Recursos compartidos

El sistema tiene tres recursos compartidos y cada uno cuenta con su propio mecanismo de protección. La Tabla 2 los resume junto con los hilos que acceden a ellos.

Tabla 2. Recursos compartidos y mecanismo de protección

Recurso	Hilos que acceden	Protección	Riesgo si no se protege
Cola de peticiones (`RequestQueue`)	Dispatcher y todos los Workers	`mtx_` + `notFull_` + `notEmpty_`	Pérdida o duplicación de peticiones, corrupción de la estructura
Contador de entradas (`TicketInventory`)	Todos los Workers	`mtx_` propio (se omite con `--no-sync`)	Sobreventa por el patrón verificar y luego actuar
Consola (`Logger`)	Todos los hilos	`mtx_` propio	Mensajes mezclados e ilegibles

Fuente: elaboración propia.

Las estadísticas de cada Worker no aparecen en la tabla porque no se comparten. Cada Worker escribe solo las suyas y el hilo principal las lee después de `join()`, que actúa como punto de sincronización.

5.3. Flujo de ejecución

La ejecución empieza leyendo los parámetros y creando la cola, el inventario y el Logger. Luego se lanzan los Workers, que se bloquean de inmediato porque la cola está vacía, y por último se lanza el Dispatcher.

El Dispatcher genera las peticiones con un Mersenne Twister. Matsumoto y Nishimura [15] presentaron este algoritmo en 1998 en *ACM Transactions on Modeling and Computer Simulation*, y C++ lo ofrece como `std::mt19937`. Con una semilla fija se repite la misma secuencia.

Cuando la cola se llena, el Dispatcher espera en `notFull_`. Cada vez que un Worker extrae una petición, avisa por esa misma condición. A la inversa, los Workers esperan en `notEmpty_` y el Dispatcher los despierta al insertar.

5.4. Protocolo de terminación

Al terminar de generar, el Dispatcher llama a `close()`, que activa la bandera `closed_` y ejecuta `notify_all` sobre ambas condiciones. Los Workers siguen consumiendo lo que quede en la cola y solo terminan cuando la encuentran cerrada y vacía.

Este detalle importa: si los Workers terminaran apenas vieran la bandera, se perderían peticiones. La verificación final compara las peticiones generadas con las atendidas para detectar ese caso y devuelve el código 2 si no coinciden.

6. Implementación

6.1. Cola acotada (RequestQueue)

El corazón del sistema es el método `push` de la cola. El fragmento siguiente muestra la espera con predicado y el aviso a los consumidores, que se hace después de liberar el cerrojo para no despertar a un hilo que tendría que volver a bloquearse.

```
std::unique_lock<std::mutex> lock(mtx_);
if (queue_.size() >= capacity_ && !closed_) {
    blocked = true;
    if (onBlock) onBlock();
    notFull_.wait(lock, [this] {           // espera con predicado
        return queue_.size() < capacity_ || closed_;
    });
}
```

```
if (closed_) return PushResult{false, blocked, queue_.size()};
queue_.push(req);
lock.unlock();
notEmpty_.notify_one();                    // "hay trabajo"
```

El método pop es simétrico: espera en notEmpty_, extrae el elemento del frente y notifica en notFull_. Si la cola está cerrada y vacía devuelve got = false, que es la señal para que el Worker salga de su bucle.

Los *hooks* onBlock y onEnqueued se ejecutan con el cerrojo de la cola tomado. Así los mensajes de consola salen en el orden real de los eventos, a cambio de respetar el orden de bloqueo cola → consola descrito en la sección 3.5.

6.2. Dispatcher

El Dispatcher genera totalRequests peticiones con un usuario entre 1 y 100 y una cantidad de 1 a 4 entradas. Si tuvo que esperar por cola llena, imprime «Espacio liberado» al reanudar, lo que permite comprobar el bloqueo desde las pruebas.

6.3. Worker

Cada Worker repite un ciclo simple: toma una petición, simula el procesamiento con sleep_for durante un tiempo aleatorio entre --min-ms y --max-ms y llama a purchase. Según el resultado, cuenta la compra como confirmada o rechazada.

6.4. Inventario de entradas

TicketInventory encapsula el contador y ofrece tres estrategias, que se eligen con --sync-mode: con mutex, atómica (sin cerrojo) y sin protección. El fragmento siguiente muestra cómo purchase atiende cada modo.

```
switch (mode_) {
    case SyncMode::Mutex: {                    // sección crítica clásica
        std::unique_lock<std::mutex> lock(mtx_);
        return purchaseUnlocked(quantity);
    }
    case SyncMode::Atomic:                     // sin cerrojo (CAS)
        return purchaseAtomic(quantity);
    case SyncMode::None: break;                // SIN protección (5.8)
}
return purchaseUnlocked(quantity);  // 1) verificar 2) pausa 3) descontar
```

Esa pausa (--window-us, 200 µs por defecto) representa la validación del pago. Con el mutex no afecta la corrección, pero sin él agranda la ventana en la que otro hilo puede verificar con el mismo stock, y así la carrera se vuelve observable.

El modo atómico aplica control de concurrencia optimista: lee el stock, lo verifica y luego intenta escribir con `compare_exchange_weak`. Si otro hilo cambió el valor entre medio, la operación falla, el hilo recibe el valor actual y vuelve a verificar.

```
int cur = atomicAvailable_.load();
while (true) {
    if (cur < quantity) return {false, cur};             // 1) verificar
    paymentWindow();                                     // 2) validar pago
    if (atomicAvailable_.compare_exchange_weak(cur, cur - quantity))
        return {true, cur - quantity};                   // 3) descontar
}   // si el CAS falla, cur ya trae el valor nuevo y se reintenta
```

Herlihy y Shavit [14] llaman a este tipo de algoritmo *lock-free*: garantiza que siempre algún hilo avance, aunque uno en particular pueda reintentar varias veces. Además, `purchase` ahora rechaza las cantidades no positivas sin tocar el stock.

6.5. Logger

El Logger serializa la escritura con un mutex para que las líneas de distintos hilos no se mezclen. Ahora recibe el flujo de salida por parámetro (por defecto `std::cout`), lo que permite probarlo escribiendo en un `std::ostringstream`.

La opción `--timestamps` antepone a cada traza el tiempo transcurrido en milisegundos. La marca se toma dentro de la sección crítica, así que aparece en el mismo orden que las líneas; el texto, en cambio, se arma fuera para no alargar el tiempo con el cerrojo tomado.

6.6. Configuración y orquestación

`config.cpp` interpreta los argumentos y valida cada valor dentro de un rango; si algo es inválido, el programa termina con el código 64. `simulation.cpp` crea los objetos y los hilos, espera con `join()` y calcula el resumen y la verificación final.

6.7. Registro de eventos de sincronización

Para observar el comportamiento interno se agregó el módulo `tracer`. Con `--trace archivo.json`, cada hilo registra los eventos que importan al SO: espera y adquisición de mutex, `wait` y `notify` de las condiciones, la sección crítica del inventario y los reintentos de CAS.

La espera por un mutex se detecta de forma real: primero se intenta `try_lock` y, solo si otro hilo lo tiene, se registra la espera antes de bloquearse con `lock`. El registro usa su propio mutex como cerrojo hoja, así que no agrega riesgo de interbloqueo.

```
{"t":26820,"th":"W1","ev":"lock","obj":"inventory"}
{"t":26938,"th":"W2","ev":"lock_wait","obj":"inventory"}
{"t":42820,"th":"W1","ev":"commit","obj":"inventory","qty":1,"remaining":19}
```

6.8. Visualizador animado para la exposición

El archivo `gui/index.html` reproduce paso a paso una traza real y anima lo que ocurre (Figura 2): la cola con sus espacios, las salas de espera de `notFull` y `notEmpty`, el dueño de cada mutex, los hilos bloqueados y la sección crítica del inventario.

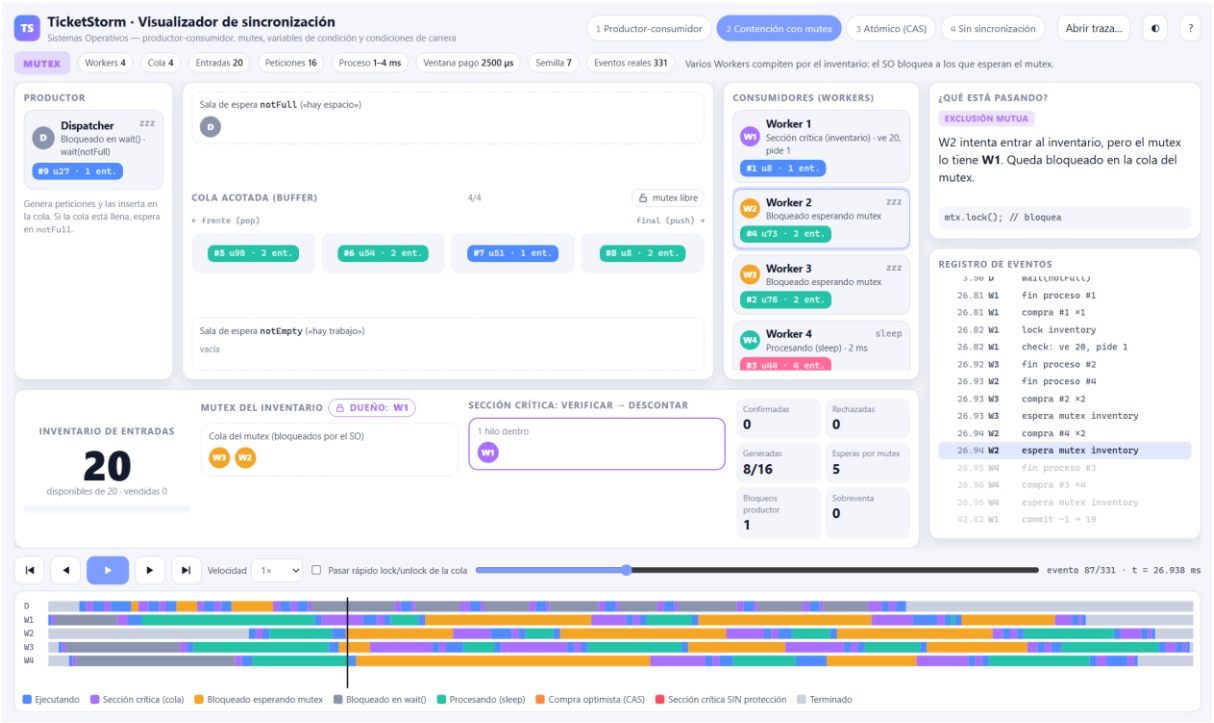


Figura 2. Visualizador: W1 tiene el mutex del inventario y W2 y W3 esperan bloqueados
Fuente: elaboración propia.

Cada evento viene con una explicación del concepto de SO y de la línea de C++ involucrada. Abajo, un diagrama de Gantt muestra el estado de cada hilo a lo largo del tiempo, al estilo de los diagramas de planificación que presenta la obra ya citada [1].

La Figura 3 muestra el escenario sin sincronización en el instante exacto de la carrera: W2 y W3 verifican el stock a la vez, ambos ven 20 entradas y la zona crítica se marca en rojo. Al final de esa ejecución se vendieron 21 entradas de 20.

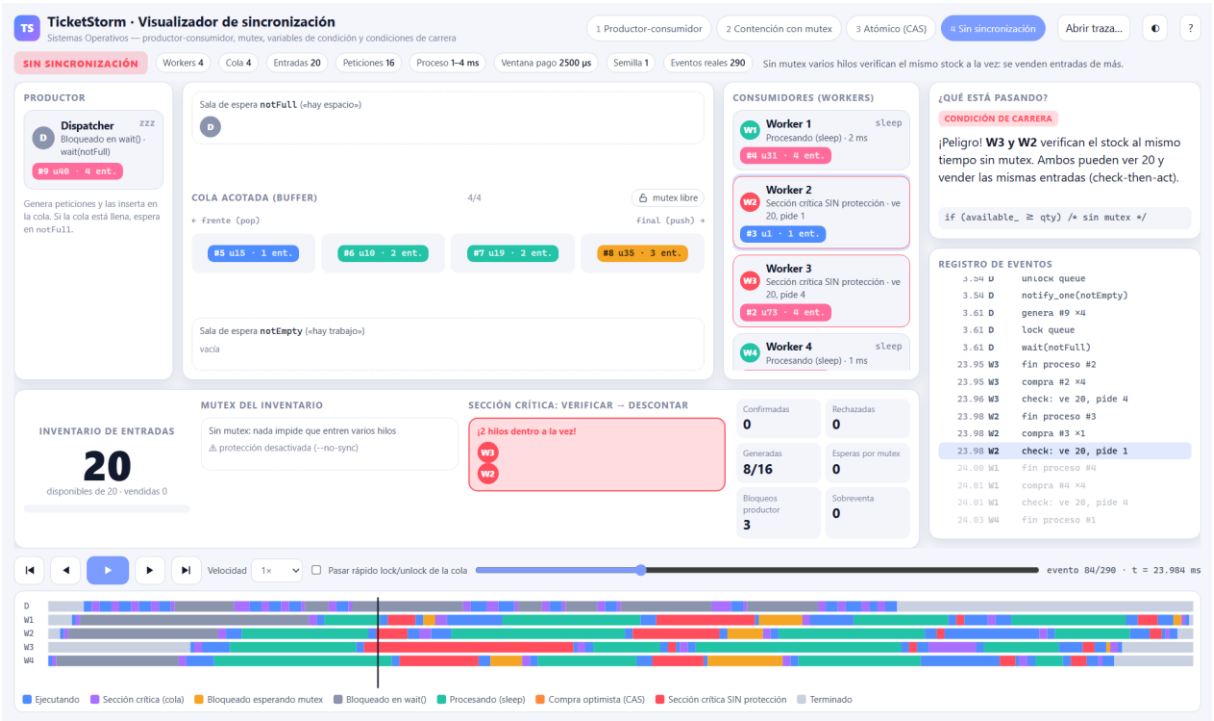


Figura 3. Visualizador en el instante de la condición de carrera (modo sin sincronización)
Fuente: elaboración propia.

El visualizador funciona sin conexión a internet y trae cuatro escenarios listos para la exposición. Se maneja con el teclado (espacio, flechas y teclas 1 a 4) y puede abrir cualquier traza nueva generada con `--trace`.

7. Compilación y ejecución

7.1. Requisitos y entorno de prueba

Se necesita un compilador con soporte de C++17 e hilos, GNU Make y Bash. El proyecto se probó en Windows 11 con MSYS2 (g++ 16.1.0, UCRT64) y GNU Make 4.4.1. El código no usa nada propio de una plataforma, así que también compila en Linux y macOS.

7.2. Comandos principales

Los objetivos del Makefile cubren la compilación, las pruebas y la limpieza. Los comandos más usados son los siguientes:

```
make                    # compila ./ticketstorm (con -Wall -Wextra -Wpedantic)
make test               # pruebas unitarias + plan de pruebas T1-T10
make unit               # solo las pruebas unitarias (U1-U10)
make bench              # benchmark mutex vs atómico
make gui                # genera las trazas del visualizador (gui/)
./ticketstorm --seed 42 # ejecución base reproducible
make tsan               # binario con ThreadSanitizer (Linux/macOS)
make clean              # elimina build/ y los binarios
```

7.3. Parámetros y códigos de salida

La Tabla 3 lista las opciones de línea de comandos. Todas tienen un valor por defecto, así que el programa puede ejecutarse sin argumentos.

Tabla 3. Opciones de línea de comandos

Opción	Descripción	Rango	Defecto
`-w, --workers N`	Número de Workers	1 – 64	3
`-q, --queue N`	Capacidad de la cola	1 – 100 000	5
`-t, --tickets N`	Entradas disponibles	1 – 10 000 000	50
`-r, --requests N`	Peticiones a generar	1 – 10 000 000	30
`--min-ms N` / `--max-ms N`	Tiempo de procesamiento por petición (ms)	0 – 60 000	50 / 150
`--window-us N`	Pausa entre verificar y descontar (µs)	0 – 1 000 000	200
`--seed N`	Semilla (0 = aleatoria)	0 – 4 294 967 295	0
`--sync-mode M`	Estrategia del contador: mutex, atomic o none	—	mutex
`--atomic` / `--no-sync`	Atajos de `--sync-mode atomic` y `none`	—	no
`--quiet`	Solo cabecera y resumen	—	no
`--timestamps`	Marca de tiempo en cada traza	—	no
`--trace ARCHIVO`	Guarda los eventos de sincronización en JSON	—	—

Fuente: elaboración propia.

El código de salida resume el resultado de la verificación, lo que permite automatizar las pruebas desde un script. La Tabla 4 muestra su significado.

Tabla 4. Códigos de salida del programa

Código	Significado
0	Ejecución correcta: vendidas + restantes = stock inicial y no hubo sobreventa.
1	Condición de carrera detectada (sobreventa o inconsistencia en el stock).
2	Peticiones perdidas: atendidas ≠ generadas.
64	Argumentos inválidos.

Fuente: elaboración propia.

8. Plan de pruebas y resultados

8.1. Metodología

Las pruebas están automatizadas en `scripts/run_tests.sh` y se ejecutan con `make test`. Cada caso lanza el binario con parámetros concretos y revisa tanto el código de salida como los mensajes esperados en la consola.

Como los errores de concurrencia no siempre se manifiestan, las pruebas críticas se repiten varias veces, siguiendo la idea comentada en la obra de Herlihy y Shavit [14]. En la revisión se añadieron los casos T7 a T10 y diez pruebas unitarias (U1–U10) del inventario y el logger con hasta 16 hilos.

8.2. Resultados

La Tabla 5 muestra cada caso de prueba con su resultado antes y después de las correcciones descritas en la sección 9.

Tabla 5. Plan de pruebas y resultados obtenidos

ID	Objetivo	Configuración	Criterio de éxito	Antes	Después
T1	Ejecución base	`--seed 42`	Código 0 y «Verificación: OK»	Falla	Pasa
T2	Bloqueo del productor	`-w 1 -q 1 -r 12 --seed 7`	Aparecen «Cola llena» y «Espacio liberado»	Falla	Pasa
T3	Espera pasiva	`-w 16 -q 5 -r 20 --seed 3`	Aparece «Cola vacía» y bajo uso de CPU	Falla	Pasa
T4	Carrera sin mutex	`--no-sync -w 8 -t 100 -r 2000`	Sobreventa detectada (código 1) en ≤ 20 intentos	Pasa	Pasa
T5	Corrección con mutex	`-w 8 -t 100 -r 2000` × 5	Ninguna sobreventa	Pasa	Pasa
T6	Repetibilidad	`--seed 42` × 10	10 verificaciones OK	Falla	Pasa
T7	Validación de argumentos	11 entradas inválidas	Código 64 en todas	—	Pasa
T8	Regresión de `--seed`	`--seed 4294967295`	Semilla aceptada y aplicada	—	Pasa
T9	Modo atómico	`--atomic -w 8 -t 100 -r 2000` × 5	Ninguna sobreventa	—	Pasa
T10	Marcas de tiempo	`--timestamps --seed 5`	Trazas con tiempo en ms	—	Pasa
U1–U10	Pruebas unitarias	`make unit` (10 113 comprobaciones)	Ninguna comprobación fallida	—	Pasa

Fuente: elaboración propia.

La columna «Antes» corresponde al código original compilado en Windows. Cuatro de las seis pruebas fallaban, pero no por un problema de concurrencia, sino porque el programa rechazaba la opción `--seed`, como se explica en la sección 9.

Tras las correcciones pasan las 10 pruebas de integración y las 10 unitarias. En T3, con 16 Workers, el tiempo real fue de 0,275 s y el tiempo de CPU de usuario de solo 0,015 s, lo que confirma que los hilos ociosos esperan bloqueados y no en un bucle activo.

En T2 se ve el comportamiento esperado del buffer acotado: con un solo Worker y capacidad 1, el Dispatcher alterna entre «Cola llena, esperando espacio...» y «Espacio liberado» en cada petición. El Anexo A muestra una salida completa de ejemplo.

9. Errores detectados, correcciones y mejoras

Durante la revisión se compiló el proyecto con todas las advertencias activas (`-Wall -Wextra -Wpedantic`) sin obtener ningún aviso. Los problemas aparecieron al ejecutar el programa y las pruebas en Windows, y se resumen en la Tabla 6.

Tabla 6. Errores detectados y corrección aplicada

ID	Archivo	Problema	Impacto	Corrección
E1	`src/config.cpp`	`long` de 32 bits en Windows trunca el límite de `--seed` a −1	Alto: ninguna semilla es aceptada; fallan T1, T2, T3 y T6	Usar `long long` y `std::stoll`
E2	`Makefile`	El objetivo se llama `ticketstorm` pero se genera `ticketstorm.exe`	Bajo: se reenlaza en cada `make`	Sufijo `$(EXE)` si `OS=Windows_NT`
E3	`scripts/run_tests.sh`	Rutas fijas en `/tmp`	Medio: choques entre ejecuciones simultáneas y poca portabilidad	Directorio con `mktemp` y limpieza con `trap`
E4	Repositorio	Sin política de fin de línea	Medio: un checkout con CRLF rompe el script en Bash	`.gitattributes` con `eol=lf`
E5	`tests/test_inventory_logger.cpp`	Contadores de comprobaciones `int` modificados desde varios hilos	Bajo: el total variaba entre ejecuciones (10 101 frente a 10 113)	`std::atomic<int>`

Fuente: elaboración propia.

9.1. Truncamiento de long en Windows

El error más grave estaba en `parseInt`. La función usaba `long` para validar rangos y el límite de `--seed` se escribía como `4294967295L`. En Linux `long` tiene 64 bits, pero en Windows (modelo LLP64) tiene solo 32, así que el límite se convertía en −1.

El síntoma era el mensaje «El valor de --seed debe estar entre 0 y -1», que hacía imposible fijar una semilla. La corrección fue usar `long long` y `std::stoll`, que según la norma [7] tienen al menos 64 bits en cualquier plataforma.

```
- bool parseInt(..., long lo, long hi, long& out, ...)
-     const long value = std::stol(text, &pos);
+ bool parseInt(..., long long lo, long long hi, long long& out, ...)
+     const long long value = std::stoll(text, &pos);
- parseInt(arg, value, 0, 4294967295L, n, error)
+ parseInt(arg, value, 0, 4294967295LL, n, error)
```

9.2. Portabilidad del build y de las pruebas

En Windows el enlazador agrega la extensión `.exe`, mientras que el Makefile esperaba un archivo llamado `ticketstorm`, por lo que `make` volvía a enlazar siempre. Se añadió el sufijo `$(EXE)` cuando la variable `OS` vale `Windows_NT`.

El script de pruebas guardaba salidas en rutas fijas de `/tmp`. Ahora usa un directorio creado con `mktemp` y eliminado con `trap`, lo que además evita choques si corren dos baterías a la vez. Un archivo `.gitattributes` fija los saltos de línea LF.

9.3. Limitación observada

ThreadSanitizer no está disponible en el compilador MinGW de Windows, así que `make tsan` debe ejecutarse en Linux o macOS. No es un error del código, pero conviene tenerlo presente al reproducir la prueba de la versión sin mutex.

El error E5 apareció en el propio arnés de pruebas unitarias: la macro CHECK se usa dentro de hilos concurrentes y sus contadores eran enteros normales. Fue una carrera de datos real, detectada porque el total de comprobaciones cambiaba entre ejecuciones.

9.4. Control de versiones

Chacon y Straub [16], en su libro *Pro Git* (2014), recomiendan aislar cada cambio en una rama temática y luego integrarla en la rama principal. Se siguió ese flujo, con mensajes de commit del tipo `tipo(ámbito): descripción`.

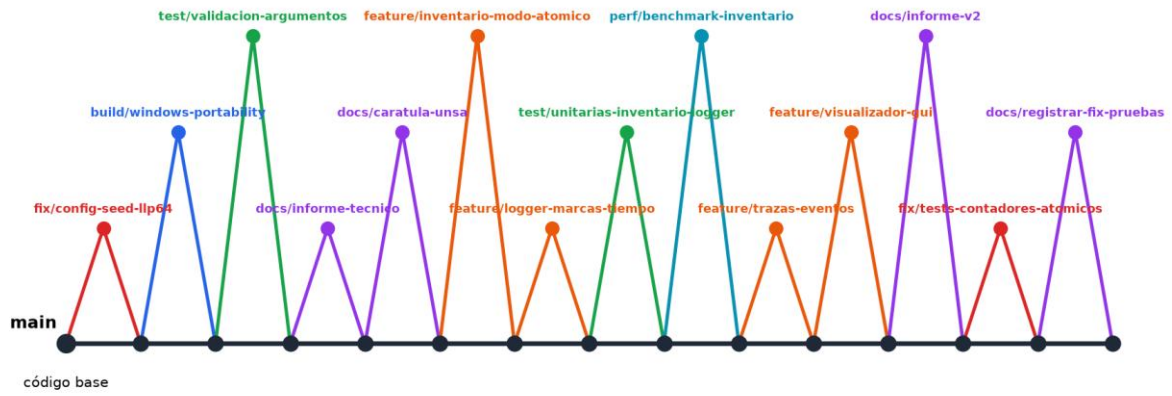


Figura 4. Historial de ramas del repositorio

Fuente: elaboración propia.

Tabla 7. Ramas creadas y propósito de cada una

Rama	Tipo	Propósito
`main`	Principal	Código estable; recibe cada rama mediante merge --no-ff
`fix/config-seed-llp64`	fix	Corrige el truncamiento de `long` en la validación de argumentos (E1)
`build/windows-portability`	build	Makefile con `.exe`, script con `mktemp` y `.gitattributes` (E2–E4)
`test/validacion-argumentos`	test	Agrega las pruebas T7 y T8
`docs/informe-tecnico`	docs	Agrega el README y este informe
`docs/caratula-unsa`	docs	Completa la carátula con los datos de la UNSA
`feature/inventario-modo-atomico`	feature	Modo atómico con `compare_exchange` y validaciones
`feature/logger-marcas-tiempo`	feature	Marcas de tiempo y salida inyectable en el Logger
`test/unitarias-inventario-logger`	test	Pruebas U1–U10, T9 y T10
`perf/benchmark-inventario`	perf	Benchmark mutex vs atómico
`feature/trazas-eventos`	feature	Registro de eventos con `--trace`
`feature/visualizador-gui`	feature	Visualizador animado para la exposición
`docs/informe-v2`	docs	Actualiza este informe y el README
`fix/tests-contadores-atomicos`	fix	Corrige la carrera en el arnés de pruebas (E5)
`docs/registrar-fix-pruebas`	docs	Registra el error E5 en este informe

Fuente: elaboración propia.

Como se ve en la Figura 4 y la Tabla 7, las ramas se integraron con `merge --no-ff` para que el historial registre qué commits pertenecían a cada cambio. Así es sencillo revisar o revertir una corrección completa más adelante.

9.5. Mejoras a los módulos de inventario y logger

Además de corregir errores, se mejoraron los módulos `ticket_inventory` y `logger`, a cargo del integrante 4, y se agregó el módulo `tracer`. La Tabla 8 resume cada mejora y el beneficio que aporta.

Tabla 8. Mejoras introducidas en los módulos

Módulo	Mejora	Beneficio
`ticket_inventory`	Enum `SyncMode` (mutex, atomic, none)	Reemplaza la bandera booleana y permite comparar tres estrategias
`ticket_inventory`	Modo atómico con `compare_exchange_weak`	Exclusión sin cerrojo: ningún hilo se bloquea
`ticket_inventory`	Rechazo de cantidades no positivas	Protege la invariante $0 \leq$ stock $\leq$ total
`logger`	Flujo de salida inyectable	Permite pruebas unitarias sin usar la consola
`logger`	Marcas de tiempo (`--timestamps`)	Muestra el orden y la duración real de los eventos
`logger`	Contador `linesWritten()`	Comprueba en las pruebas que no se pierden líneas
`tracer` (nuevo)	Registro de eventos (`--trace`)	Alimenta el visualizador con datos reales

Fuente: elaboración propia.

10. Análisis de resultados

10.1. Efecto del mutex en la consistencia

Para medir el efecto de la sincronización se ejecutó 20 veces cada versión con 8 Workers, 100 entradas, 2000 peticiones y sin tiempo de procesamiento. La Figura 5 compara las entradas vendidas en cada ejecución.

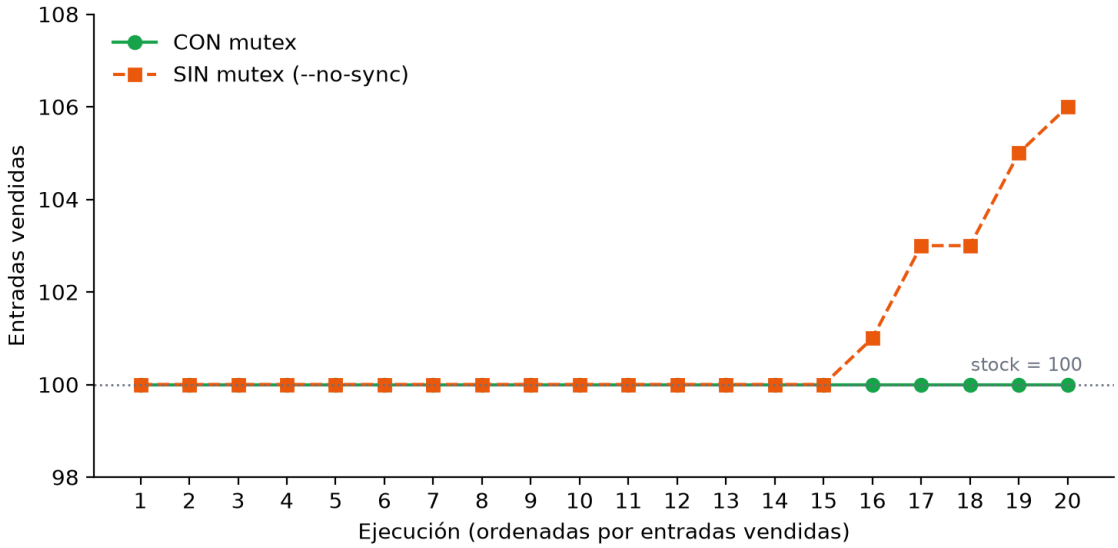


Figura 5. Entradas vendidas en 20 ejecuciones con y sin mutex (stock de 100)

Fuente: elaboración propia.

Sin mutex, 5 de las 20 ejecuciones (25 %) vendieron más entradas de las disponibles: 101, 103, 103, 105 y 106. Con mutex, las 20 vendieron exactamente 100 y la verificación fue correcta, tal como predice la obra ya citada [4].

Que la carrera aparezca solo en una de cada cuatro ejecuciones muestra por qué estos errores son tan peligrosos: un programa incorrecto puede pasar muchas pruebas seguidas. Solo la sincronización garantiza la corrección, no la suerte en la intercalación.

10.2. Escalabilidad con el número de Workers

También se midió el tiempo total con 60 peticiones y entre 1 y 16 Workers, usando la semilla 42. La Tabla 9 y la Figura 6 muestran que el tiempo baja casi en proporción al número de Workers.

Tabla 9. Tiempo total según el número de Workers (60 peticiones, 50–150 ms)

Workers	Tiempo total (ms)	Aceleración	Eficiencia
1	6457	1,00	100 %
2	3380	1,91	96 %
4	1639	3,94	98 %
8	925	6,98	87 %
16	497	12,99	81 %

Fuente: elaboración propia.

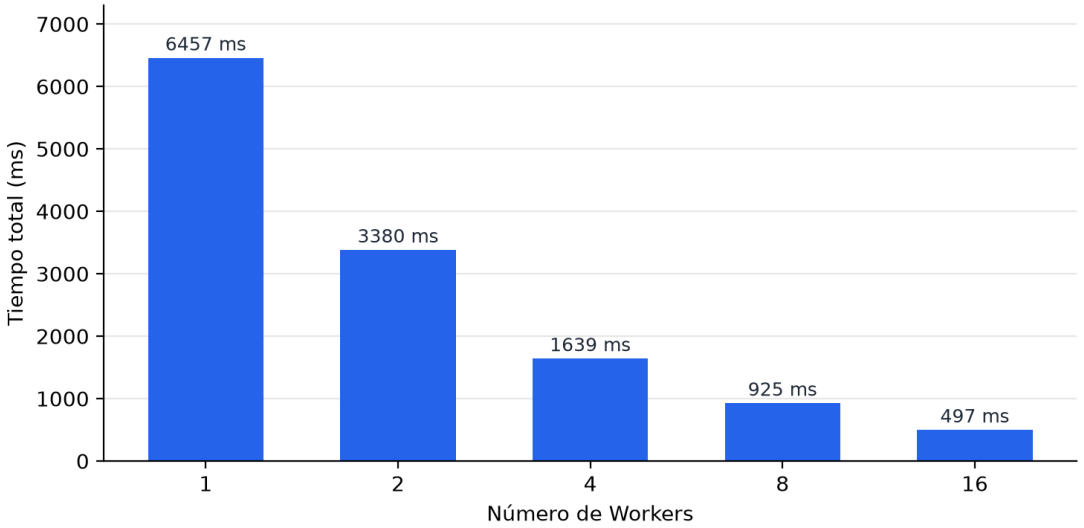


Figura 6. Tiempo total de la simulación según el número de Workers

Fuente: elaboración propia.

La aceleración es casi lineal hasta 4 Workers y luego se aleja un poco del ideal. Hay que aclarar que el procesamiento se simula con `sleep_for`, así que los Workers no compiten por CPU: lo que se mide es cuánto trabajo en espera se puede solapar.

Con 16 Workers la eficiencia baja al 81 %, porque las 60 peticiones se reparten en pocas rondas y el arranque y el cierre pesan más. Aun así, la cola acotada no se convierte en cuello de botella para estas cargas.

10.3. Mutex frente a operaciones atómicas

Con `make bench` se midió el costo de una compra en los modos mutex y atómico: 2 000 000 de compras repartidas entre 1 y 16 hilos, sin ventana de pago. La Tabla 10 y la Figura 7 muestran la mediana de cinco repeticiones en un Intel Core i5-12450H de 12 hilos lógicos.

Tabla 10. Costo por compra en modo mutex y atómico (ns por operación)

Hilos	Mutex	Atómico (CAS)	Atómico / mutex
1	24,9	26,1	1,05
2	47,9	63,4	1,32
4	35,4	79,7	2,25
8	62,9	95,5	1,52
16	67,6	118,3	1,75

Fuente: elaboración propia.

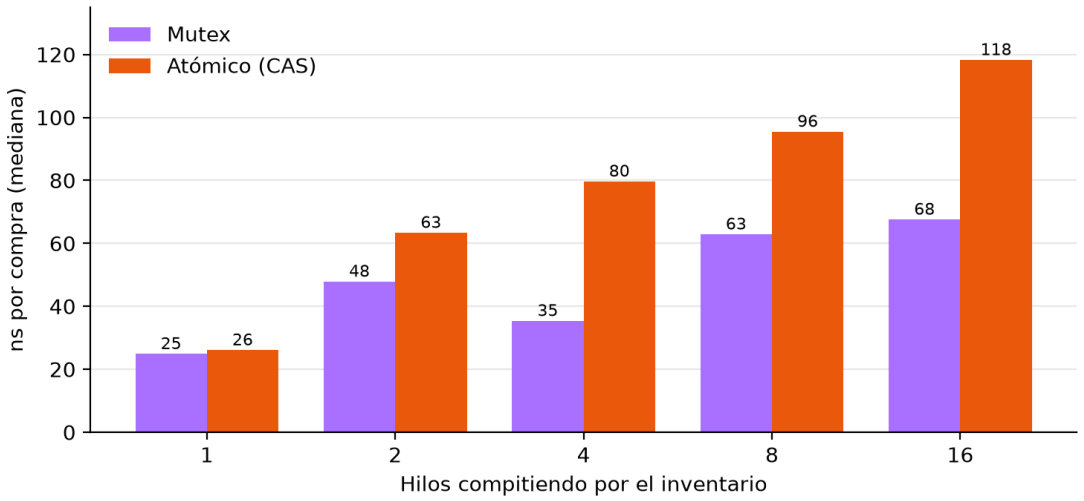


Figura 7. Costo por compra según el número de hilos (mediana de 5 repeticiones)

Fuente: elaboración propia.

El resultado contradice la idea de que lo atómico siempre es más rápido. Con un hilo ambos cuestan casi lo mismo, pero con contención el CAS falla y reintenta, y la línea de caché del contador viaja de un núcleo a otro en cada intento.

El mutex, en cambio, duerme a los hilos que no pueden entrar, así que en cada instante compiten menos. Williams [6] advierte que una estructura sin cerrojos no es necesariamente más rápida; un contador único muy disputado, como este, es justo el peor caso.

10.4. Resolución del temporizador del SO

Las trazas revelaron además un detalle del planificador de Windows: una pausa de 2500 µs pedida con `sleep_for` duró cerca de 16 ms, porque el SO despierta a los hilos según la resolución de su temporizador. Tanenbaum y Bos [2] explican cómo esos ticks del reloj marcan ese límite.

11. Conclusiones

El modelo productor-consumidor con una cola acotada, un mutex y dos variables de condición coordinó al Dispatcher y a los Workers sin espera activa, como se comprobó en la prueba T3 con un uso de CPU casi nulo.

La comparación con y sin mutex demostró en la práctica la condición de carrera del patrón verificar y luego actuar: sin sincronización hubo sobreventa en el 25 % de las ejecuciones, y con ella en ninguna.

La bandera `closed_` combinada con `notify_all` permitió una terminación limpia, sin peticiones perdidas, en todas las ejecuciones realizadas.

La revisión mostró que la portabilidad también es parte de la corrección: un detalle del modelo de datos de Windows dejaba inutilizable la opción `--seed` y hacía fallar cuatro de las seis pruebas originales.

El modo atómico también evitó la sobreventa, pero con contención fue más lento que el mutex. Esto muestra que la mejor primitiva de sincronización depende de cuántos hilos compiten por el mismo dato.

El visualizador construido sobre trazas reales convierte conceptos abstractos, como el `wait` de una variable de condición o una condición de carrera, en algo que se puede observar paso a paso durante la exposición.

El uso de ramas temáticas y commits descriptivos dejó un historial claro de qué se cambió, por qué y en qué orden, lo que facilita la revisión por parte de todo el equipo.

12. Recomendaciones y trabajo futuro

- Ejecutar `make tsan` en Linux como parte de una integración continua, para detectar carreras de datos en cada cambio.
- Agregar pruebas unitarias de `RequestQueue` y del Dispatcher, siguiendo el estilo de `tests/test_inventory_logger.cpp`.
- Probar un contador repartido por Worker (*sharding*) para reducir la contención sobre una única línea de caché.
- Permitir que el visualizador lance el programa con parámetros elegidos en vivo mediante un pequeño servidor local.
- Medir el sistema con procesamiento que use CPU real en lugar de `sleep_for`, para estudiar la escalabilidad respecto al número de núcleos.

Referencias

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.

[3] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Hoboken, NJ, USA: Pearson, 2018.

[4] E. W. Dijkstra, "Cooperating sequential processes," Technological Univ. Eindhoven, Eindhoven, The Netherlands, Tech. Rep. EWD-123, 1965.

[5] H.-J. Boehm and S. V. Adve, "Foundations of the C++ concurrency memory model," in *Proc. 29th ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, Tucson, AZ, USA, 2008, pp. 68–78.

[6] A. Williams, *C++ Concurrency in Action*, 2nd ed. Shelter Island, NY, USA: Manning, 2019.

[7] *Programming Languages — C++*, ISO/IEC Standard 14882:2017, 2017.

[8] C. A. R. Hoare, "Monitors: An operating system structuring concept," *Commun. ACM*, vol. 17, no. 10, pp. 549–557, Oct. 1974.

[9] B. W. Lampson and D. D. Redell, "Experience with processes and monitors in Mesa," *Commun. ACM*, vol. 23, no. 2, pp. 105–117, Feb. 1980.

[10] D. R. Butenhof, *Programming with POSIX Threads*. Reading, MA, USA: Addison-Wesley, 1997.

[11] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, v1.00. Madison, WI, USA: Arpaci-Dusseau Books, 2018.

[12] E. G. Coffman, M. J. Elphick, and A. Shoshani, "System deadlocks," *ACM Comput. Surv.*, vol. 3, no. 2, pp. 67–78, Jun. 1971.

[13] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: Data race detection in practice," in *Proc. Workshop Binary Instrum. Appl. (WBIA)*, New York, NY, USA, 2009, pp. 62–71.

[14] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Burlington, MA, USA: Morgan Kaufmann, 2008.

[15] M. Matsumoto and T. Nishimura, "Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator," *ACM Trans. Model. Comput. Simul.*, vol. 8, no. 1, pp. 3–30, Jan. 1998.

[16] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Berkeley, CA, USA: Apress, 2014.

Anexo A. Salida de ejemplo

Fragmento de la salida de `./ticketstorm --seed 42` con la configuración por defecto (3 Workers, cola de 5, 50 entradas y 30 peticiones).

```
=== TicketStorm ===
Workers: 3 | Capacidad de cola: 5 | Entradas: 50 | Peticiones: 30
Modo: CON sincronización (mutex) | Semilla: 42
[Worker 1] Cola vacía, esperando trabajo...
[Worker 3] Cola vacía, esperando trabajo...
[Dispatcher] Petición #1: Usuario 38 solicita 4 entradas (cola: 1/5)
[Dispatcher] Petición #2: Usuario 96 solicita 1 entradas (cola: 2/5)
[Worker 3] Despertado: hay trabajo disponible.
[Dispatcher] Petición #6: Usuario 16 solicita 1 entradas (cola: 5/5)
[Dispatcher] Cola llena, esperando espacio...
[Dispatcher] Espacio liberado, se reanuda la generación.
[Worker 3] Compra confirmada (Petición #1, Usuario 38, 4 entradas). Entradas restantes: 46
...
=== RESUMEN ===
Worker 1: 10 peticiones (confirmadas: 8, rechazadas: 2, entradas vendidas: 18)
Worker 2: 9 peticiones (confirmadas: 6, rechazadas: 3, entradas vendidas: 14)
Worker 3: 11 peticiones (confirmadas: 8, rechazadas: 3, entradas vendidas: 18)
Peticiones generadas: 30 | Peticiones atendidas: 30
Entradas vendidas: 50 | Peticiones rechazadas: 8
Entradas restantes: 0 (stock inicial: 50)
Verificación: OK (vendidas + restantes = stock inicial; ...)
```

Anexo B. Salida del plan de pruebas

Resultado de `make test` después de aplicar las correcciones, en el entorno descrito en la sección 7.1.

```
T1. Ejecución base (3 Workers, cola de 5)             [PASS]
T2. Cola pequeña con pocos Workers                    [PASS]
T3. Muchos Workers                                    [PASS]
    tiempo real=0.275 s | CPU usuario=0.015 s | CPU sistema=0.000 s
T4. Versión SIN mutex en el contador                  [PASS]
    Se vendieron 101 entradas habiendo solo 100 disponibles.
T5. Versión CON mutex                                 [PASS]
T6. Repetir 10 ejecuciones                            [PASS]
T7. Validación de argumentos                          [PASS]
T8. Semilla máxima de 32 bits                         [PASS]
T9. Modo atómico (compare_exchange)                   [PASS]
T10. Marcas de tiempo en las trazas                   [PASS]
Resultado: 10 pruebas correctas, 0 fallidas.

Pruebas unitarias: U1-U10 [PASS] -- Comprobaciones: 10113, fallidas: 0
```

Anexo C. Historial de commits

Historial de `main` obtenido con `git log --oneline --first-parent`: cada línea es la integración de una rama temática.

```
Merge branch 'docs/registrar-fix-pruebas'
Merge branch 'fix/tests-contadores-atomicos'
Merge branch 'docs/informe-v2'
Merge branch 'feature/visualizador-gui'
Merge branch 'feature/trazas-eventos'
Merge branch 'perf/benchmark-inventario'
Merge branch 'test/unitarias-inventario-logger'
Merge branch 'feature/logger-marcas-tiempo'
```

```
Merge branch 'feature/inventario-modo-atomico'
Merge branch 'docs/caratula-unsa'
Merge branch 'docs/informe-tecnico'
Merge branch 'test/validacion-argumentos'
Merge branch 'build/windows-portability'
Merge branch 'fix/config-seed-llp64'
chore: importar código base de TicketStorm
```

Anexo D. Guía rápida para la exposición

Abrir `gui/index.html` en el navegador y presionar F para pantalla completa. Se sugiere recorrer los escenarios en este orden:

- Escenario 1 (tecla 1): el productor se bloquea en `notFull` y los consumidores en `notEmpty`.
- Escenario 2 (tecla 2): contención por el mutex del inventario y cola de hilos bloqueados por el SO.
- Escenario 3 (tecla 3): reintentos de `compare_exchange` sin que ningún hilo se bloquee.
- Escenario 4 (tecla 4 o enlace `index.html#nosync@83`): instante exacto de la carrera y sobreventa final.
- Traza en vivo: ejecutar `./ticketstorm --trace demo.json` y arrastrar el archivo al visualizador.